#42DAYSOFML CHALLENGE

Week 5
Revision Guide

NLP and LLMs · Days 33 – 39

Concepts · Code · Real World Problems · Interview Prep

WHAT'S INSIDE

DAY 32 RECAP · BEFORE WE BEGIN

Week 4 took you from raw preprocessing into deep learning: Pipelines, Neural Networks, CNNs, Transfer Learning, RNNs and LSTMs, Transformers, and Backpropagation and Optimizers. Here is the one-line summary of each day, so Week 5's move into NLP and LLMs builds on solid ground.

Day	Topic	One-Line Takeaway
25	Pipelines	Chain preprocessing and model into one object. Prevents CV leakage, deployment scaler bugs, and train/serve skew.
26	Neural Networks	A layer is a weighted sum plus a non-linear activation. ReLU beats sigmoid in hidden layers because its gradient never vanishes.
27	CNNs	Convolution shares weights across the whole image. Pooling gives translation invariance. Both beat fully connected layers on spatial data.
28	Transfer Learning	Freeze a pretrained backbone and reuse its features. Fine-tune only when you have enough data and the domain has shifted.
29	RNNs and LSTMs	Vanishing gradients kill plain RNNs on long sequences. The LSTM cell state uses additive updates instead of repeated multiplication to fix it.
30	Transformers	Self-attention lets every token look at every other token directly, no recurrence needed. Cost scales $O(n^2)$ with sequence length.
31	Backprop and Optimizers	Batch size and learning rate move together. Gradient clipping caps the update size so one bad batch cannot blow up training.

WEEK 5 ROADMAP

Week 4 ended with Transformers, the architecture behind every modern LLM. Week 5 goes one level up the stack: how text becomes numbers a model can use (Tokenization and Embeddings), how to adapt a model to a task (Fine-tuning vs Prompting), how to ground a model in facts it was never trained on (RAG), how to let a model take actions (LLM Agents), what stores and

searches those embeddings at scale (Vector Databases), how to know if any of this is actually working (LLM Evaluation), and how to wire it all together against a real API (Building with APIs). By Day 39 you can build and evaluate a working LLM application end to end, not just call an API and hope.

SELF-CHECK BEFORE STARTING WEEK 5

Without notes: explain why a scaler belongs inside a Pipeline instead of being applied separately. State what problem the LSTM's additive cell-state update solves. Describe why self-attention needs positional encoding when recurrence does not. If any of these feel shaky, revisit that day before continuing.

DAY 33 · TOKENIZATION AND EMBEDDINGS

THE CORE IDEA

Every neural network is a function that takes numbers and returns numbers. Raw text is not numbers. Tokenization converts text into a sequence of integer IDs a model can process, and the strategy you pick decides how the model handles rare words, how long a sequence it can hold, and how many tokens a sentence costs at inference time.

Once text is tokenized, each ID is just an integer with no relationship to any other ID. Embeddings fix this by mapping each token to a dense vector learned during training. Words that show up in similar contexts end up with similar vectors, which is why vector similarity ends up tracking meaning.

THREE TOKENIZATION STRATEGIES

Strategy	How It Splits	Problem It Solves	Problem It Creates
Character-level	Every character is a token	No unknown words, ever	Very long sequences, no word-level meaning
Word-level	Every word is a token	Captures word meaning directly	OOV problem: unseen words become [UNK]
Subword (BPE)	Common sequences are merged	Handles rare words by splitting into known parts	Slightly less intuitive to read

FROM THE DAY 33 NOTEBOOK · TOKEN COUNT ON THE SAME SENTENCE

Sentence: "The transformer architecture changed NLP in 2017"

Strategy	Token Count	Verdict
Character-level	55 tokens	Too long, no word meaning
Word-level	7 tokens	Good length, but breaks on unseen words
BPE subword	7 tokens	Best of both. This is what GPT-4 uses.

BPE IN THREE STEPS

```
1. Start: split every word into characters + an end-of-word marker </w>
2. Count: find the most frequent adjacent pair across the corpus
3. Merge: replace every occurrence of that pair with a combined token
   Repeat steps 2-3 for N merges (GPT-2 uses 50,000 merges)
```

COSINE SIMILARITY: MEASURING MEANING, NOT JUST WORDS

```
cosine_similarity(a, b) = (a . b) / (||a|| * ||b||)
```

```
Range: -1 (opposite)  0 (unrelated)  1 (identical direction)
```

Toy embeddings built from just 4 dimensions (royalty, gender, France, UK) already show the pattern real models learn at scale: king and queen cluster on the royalty axis, paris and france cluster on geography, and dog and cat sit far from both clusters. Sentence embeddings from all-MiniLM-L6-v2 extend the same idea to full sentences: 'The dog chased the cat' and 'A feline was pursued by a canine' score high on cosine similarity despite sharing zero words.

PRODUCTION TRAP · RE-EMBEDDING EVERYTHING ON A MODEL UPGRADE

In November 2023, OpenAI replaced text-embedding-ada-002 with text-embedding-3-small. The new model produces a different vector space, so old and new embeddings cannot be compared against each other. A startup with 500,000 ada-002 embeddings had two options: re-embed everything (roughly $5 in API fees, but a schema rewrite and redeploy) or stay behind on retrieval quality.

Engineers who had stored the source text alongside each embedding re-embedded in one afternoon. Engineers who had only stored the vectors had to reconstruct their document pipeline from scratch first.

Lesson: always store the source text next to the embedding. Embeddings are reproducible. Lost source text is not.

GOLDEN RULE

Embeddings encode distributional meaning, not word identity. Two sentences with zero words in common can score 0.95 on cosine similarity because the model learned they play the same role in similar contexts. Matryoshka Representation Learning (OpenAI's text-embedding-3 models) takes this further: the first 256 dimensions of a 1536-dim embedding are themselves a valid embedding, which is why production teams run a cheap 256-dim first pass for retrieval and a full 1536-dim pass for re-ranking the top results.

DAY 34 · FINE-TUNING VS PROMPTING

THE CORE IDEA

Prompting keeps the model weights frozen. You write instructions, examples, and context into the text you send, and the model uses its existing knowledge to follow them. Fine-tuning updates the weights themselves, training the model on new examples so it gets better at one specific task. Neither is always the right call. The choice depends on your data volume, your budget, your format requirements, and how often the task changes.

THREE PROMPTING TECHNIQUES, BEFORE YOU EVEN CONSIDER FINE-TUNING

Technique	What You Give the Model	Works Best For
Zero-shot	Task description only	Simple, well-defined tasks the model already has strong priors on
Few-shot	Task description + labelled examples	Tasks needing a specific format or terminology
Chain-of-Thought	Ask the model to reason step by step first	Multi-step reasoning, math, logic

Wei et al. (2022) documented that Chain-of-Thought prompting raised accuracy on grade-school math problems from 18% zero-shot to 57%, without touching a single model weight.

WHEN FINE-TUNING ACTUALLY EARNS ITS COST

Signal	Why It Points to Fine-tuning
Consistent output format required	Prompting a strict JSON schema still fails 2-5% of the time. Thousands of correct examples push that near zero.

Signal	Why It Points to Fine-tuning
Enough labelled examples	100+ minimum for classification, 500+ for generation. Below that the model memorises instead of generalising.
Domain-specific language	Legal, medical, or financial terms are low-frequency tokens to a general model. Fine-tuning makes them first-class.
Latency or cost at scale	A fine-tuned small model with a short prompt beats a large model with a long few-shot prompt on cost and speed at high volume.

MONTHLY COST AT 50,000 CALLS

Approach	Tokens/Call	Monthly Inference	One-time Fine-tune
GPT-4 zero-shot	150 in + 50 out	highest of the four	$0
GPT-4 few-shot (8 examples)	800 in + 50 out	higher still	$0
GPT-3.5 few-shot (8 examples)	800 in + 50 out	lowest ongoing cost, no training	$0
GPT-3.5 fine-tuned (2-shot)	200 in + 50 out	low ongoing cost	~$4 (500K training tokens)

The fine-tuned option wins on unit economics at high volume, but only if the task stays stable long enough to earn back the training cost.

REAL WORLD PROBLEM · THE MODEL IMPROVEMENT TRAP

In 2022, a fintech company fine-tuned GPT-3 to extract structured data from bank statements. Cost: $40K in compute and 3 months of engineering time. Accuracy: 87%.

In March 2023, GPT-4 launched. Zero-shot on the same benchmark: 84% accuracy, zero fine-tuning cost. Six months later, a 5-shot prompt on GPT-4 hit 91%. The fine-tuned GPT-3 model was now the worst-performing option on the table, and the 3-month investment was obsolete.

Lesson: benchmark the latest base model before committing to fine-tuning. Build your evaluation harness first, so you can re-check that decision in an afternoon instead of never checking it at all.

GOLDEN RULE

Full fine-tuning updates every parameter. LoRA (Hu et al., 2021) freezes the original weights and trains two small low-rank matrices whose product approximates the weight update, cutting trainable parameters on a 7B model from 7 billion to roughly 50 million with minimal accuracy loss. This is why almost every open-source fine-tuned LLM on HuggingFace Hub was trained with LoRA or its 4-bit successor, QLoRA. Re-evaluate any fine-tuning decision every 6 months. Base models improve faster than most teams expect.

DAY 35 · RAG (RETRIEVAL-AUGMENTED GENERATION)

THE CORE IDEA

An LLM's knowledge is frozen at training time. Ask it about something outside that training data and it does not say 'I don't know,' it generates the most statistically plausible continuation of your prompt, which is often wrong. That is hallucination: confident, fluent, incorrect text. RAG fixes this by retrieving the actual relevant text at inference time and handing it to the model as context. The model reads the answer instead of recalling it from memory.

Lewis et al. (2020) showed RAG outperforming a fully fine-tuned GPT-2 on knowledge-intensive tasks, using a fraction of the parameters, because retrieval is far more parameter-efficient than memorisation.

THE RAG PIPELINE

```
OFFLINE (ingestion): Documents -> Chunking -> Embedding -> Vector DB
ONLINE (query time):  Vector DB -> Retrieval (top-k) -> Prompt (query+context) -> LLM answer
```

CHUNKING STRATEGIES

Strategy	How It Works	Problem
Fixed-size, no overlap	Split every N characters	Cuts sentences mid-thought at the boundary
Fixed-size, with overlap	Split every N chars, repeat the last M in the next chunk	Reduces boundary loss but stores redundant text
Sentence-aware	Never split a sentence; group sentences up to a max size	Clean chunks, but variable size

LangChain's RecursiveCharacterTextSplitter is the production default: it tries paragraphs first, then sentences, then words, then characters.

SPARSE (TF-IDF) VS DENSE RETRIEVAL, BY QUERY TYPE

Query Type	TF-IDF (Sparse)	Dense (Sentence-Transformers)
Keyword queries	82%	79%
Paraphrase queries	51%	83%
Cross-lingual	12%	71%
Long queries	44%	78%

Based on the BEIR benchmark (Thakur et al., 2021). Sparse wins on exact keyword match, dense wins everywhere meaning matters more than wording. Production systems combine both as hybrid search.

THE RAG PROMPT PATTERN

```
Answer the question using ONLY the context provided below.
If the answer is not found in the context, say "I don't have enough information to answer this."
Do not use any prior knowledge outside the context.

Context: {context}
Question: {question}
Answer:
```

REAL WORLD PROBLEM · RAG RETURNING THE WRONG CHUNKS

A fintech company's RAG system answers loan questions at 91% on its test set, then ships and starts mixing variable-rate details into fixed-rate answers. The embedding model treats the two terms as similar because they appear in the same context, so retrieval returns both chunk types for almost any loan query.

Three fixes engineers actually use:

1. Metadata filtering: tag each chunk with its category and filter before similarity search (ChromaDB and Pinecone both support this).

2. Re-ranking: run a cross-encoder on the top-20 retrieved chunks for higher precision, at the cost of speed.

3. Query rewriting: use an LLM to turn a vague query into a specific one before embedding it. One documented case improved retrieval precision by 18% from this step alone.

A 91% test score means little if the test set doesn't reflect real production queries. Evaluate on real user queries, not synthetic ones.

GOLDEN RULE

Standard RAG retrieves chunks by similarity to the query, which fails when an answer needs combining facts spread across multiple documents; each individual chunk looks too generic to rank highly on its own. Microsoft's GraphRAG (2024) fixes this by building a knowledge graph over the corpus and traversing relationships between entities instead of searching a flat embedding index, answering multi-hop questions standard RAG cannot touch.

DAY 36 · LLM AGENTS

THE CORE IDEA

A chatbot takes a message and returns a response in one step. An agent takes a goal and runs as many steps as it needs: at each step it decides whether it has enough information to answer, or whether it needs to act first, pick a tool, run it, read the result, and decide again.

This loop is ReAct (Reasoning and Acting), proposed by Yao et al. (2022): Thought, then Action, then Observation, repeated until the model produces a Final Answer or hits a step limit. Perplexity AI's architecture is this loop. Every query triggers a search action, the result becomes an observation, and the model generates a grounded answer from it, a chatbot on the surface and an agent underneath.

EVERY AGENT NEEDS THREE THINGS

```
1. An LLM that can reason and follow the ReAct format
2. A set of tools with clear descriptions the LLM reads to pick between them
3. A loop that runs until the LLM signals a final answer or hits a step limit
```

FROM THE DAY 36 NOTEBOOK · AGENT OUTCOME DISTRIBUTION ON COMPLEX QUERIES

Outcome	Rate	Cause / Fix
Wrong tool selected	18%	Vague descriptions. Write what the tool does NOT handle, not just what it does.
Infinite loop	12%	No step limit. Always set max_steps and detect repeated identical calls.
Hallucinated tool input	22%	Validate tool inputs before execution; return clear, recoverable error messages.
Max steps exceeded	8%	Legitimate multi-step task ran out of budget; raise the limit or simplify the task.
Correct answer	40%	The baseline you're trying to improve on.

Only 40% of runs complete correctly without any guardrails. Agents are not plug-and-play.

REAL WORLD PROBLEM · THE INFINITE LOOP IN PRODUCTION

A B2B SaaS support agent had three tools (ticket lookup, knowledge base search, email sender) and no step limit. A user asked about a refund. The agent misread a date format (MM/DD vs DD/MM), decided the ticket did not exist, and retried the same failing call over and over: 847 tool calls in 4 minutes before anyone noticed the API bill spike.

Three guards that prevent this:

1. max_steps: 8-10 steps handles 95% of legitimate requests. Anything higher is almost certainly stuck.
2. Repetition detection: if the same tool is called with near-identical input twice in a row, stop and fall back to a human.
3. Per-tool call limits: give each tool a budget so one broken tool cannot consume the whole step budget.

GOLDEN RULE

Writing out reasoning before acting forces the model to commit to a plan instead of jumping straight to a tool call, and each reasoning step becomes context for the next generation. HuggingFace's smolagents keeps the entire agent loop to about 1,000 lines of Python, readable in an afternoon, and its CodeAgent (writing and running Python directly) outperforms ReAct-style string parsing on tasks that need more than two tools combined in one reasoning step.

DAY 37 · VECTOR DATABASES

THE CORE IDEA

A traditional database answers exact-match questions: WHERE price < 100. It cannot efficiently find the 10 most semantically similar sentences to a query. A vector database stores embeddings and answers a different question: which stored vectors sit closest to this query vector? Brute-force search (compute distance to every vector, then sort) is exact but takes minutes at 100 million vectors, far too slow for a real product.

Approximate Nearest Neighbor (ANN) search trades a small amount of accuracy for a large drop in latency, using a data structure that skips most vectors instead of checking each one. HNSW (Hierarchical Navigable Small World graphs) reaches sub-millisecond search across a billion vectors at over 95% recall.

FAISS INDEX TYPES

Index	Search Type	When to Use
IndexFlatIP	Exact brute-force	Under 100K vectors, accuracy is critical
IndexIVFFlat	Approximate (partitioned)	100K to 10M vectors, tunable accuracy via nprobe
IndexHNSWFlat	Approximate (graph-based)	Fast queries, no training step needed

FROM THE DAY 37 NOTEBOOK · RECALL VS LATENCY (IVF, nprobe SWEEP)

nprobe	Recall@10	Latency
1	lowest recall	fastest
8	moderate recall	fast
32	high recall	moderate
64	near-exact recall	slowest of the sweep

Increasing nprobe searches more clusters, trading latency for recall. Tune this against your own data distribution, not a rule of thumb.

REAL WORLD PROBLEM · WHAT BREAKS AT 10 MILLION VECTORS

A legal document search tool starts at 50,000 documents on IndexFlatIP. Everything works. It grows to 10 million documents over 18 months, and three things break at once:

1. Memory: IndexFlatIP stores every vector in RAM. At 384 dimensions in float32, 10M vectors need roughly 15GB. A 16GB server is nearly maxed out before new documents even land.

2. Latency: brute-force search over 10M vectors takes about 4 seconds per query on CPU, against a 200ms SLA.

3. No metadata filtering: IndexFlatIP cannot filter by jurisdiction before searching, forcing a separate database and a join step.

Fix: switch to IndexIVFPQ. Product Quantization compresses each vector from 1,536 bytes to 48 bytes, dropping memory from 15GB to under 500MB and latency to under 50ms at 95% recall. Move metadata filtering to ChromaDB or Weaviate, which support it natively.

Your prototype index choice sets your scaling ceiling. Plan the migration before you hit the wall, not after.

GOLDEN RULE

pgvector adds a vector type and HNSW/IVFFlat indexes directly inside PostgreSQL, so a filtered similarity search is a single SQL query against the same database that already holds your metadata, no separate vector store, no sync lag. For teams already running Postgres with under 5 million vectors, that's often the right call before adding a dedicated vector database at all.

DAY 38 · LLM EVALUATION

THE CORE IDEA

Traditional ML evaluation is mostly settled: a ground truth label and a metric like accuracy or RMSE. LLMs break this in three ways. There is rarely one single correct answer, so exact-match metrics penalise valid paraphrasing. Failure modes are subtle: a response can be fluent, confident, and completely wrong. And evaluation criteria are task-specific: a RAG chatbot cares about faithfulness to retrieved context, a summariser cares about conciseness, a code assistant cares about whether the code runs. Production LLM evaluation runs a stack of metrics, never just one number.

ROUGE AND ITS LIMITS

Reference: "The Eiffel Tower is located in Paris, France, and was completed in 1889."

Response Type	ROUGE-1 F1
Close paraphrase	high
Correct, different wording	noticeably lower, despite being just as correct
Vague / incomplete	lowest

ROUGE penalises a factually correct answer for using different words than the reference. This exact limitation is what pushed the field toward semantic and LLM-based evaluation.

FAITHFULNESS AND RELEVANCY ARE MEASURED SEPARATELY

Faithfulness asks whether every claim in the answer is grounded in the retrieved context. Relevancy asks whether the answer addresses the question. An answer can be highly faithful (every word pulled from context) and still low relevancy (it answers a different question than the one asked). Production systems check both, never just one.

THE LLM-AS-JUDGE PROMPT PATTERN

```
Rate the answer from 1 (worst) to 5 (best) on each dimension:
1. Faithfulness: is every claim supported by the context?
2. Relevancy: does the answer directly address the question?
3. Completeness: does it cover what the context allows?
Respond ONLY in JSON: {"faithfulness":.., "relevancy":.., "completeness":.., "reasoning":".."}
```

REAL WORLD EXERCISE · CATCHING HALLUCINATIONS IN 5 RAG ANSWERS

Of 5 question-context-answer triples from a mock support system, 2 contained a hallucination. The easy one to catch: a shipping answer claimed same-day delivery when the context said 5-7 business days, a direct contradiction. The hard one: a premium-plan answer was mostly correct but added 'a free annual hardware upgrade' that never appeared anywhere in the context.

The hard case is the dangerous one. A single fabricated claim buried inside an otherwise accurate answer is easy for a human reviewer to miss on a skim, and it's exactly why faithfulness has to be checked automatically at scale, not spot-checked by eye.

GOLDEN RULE

Run cheap proxy metrics (TF-IDF style faithfulness and relevancy checks) on every single request, since they're fast and cost nothing. Reserve LLM-as-judge for sampled batches or offline runs. It scales better than human review and catches nuance overlap metrics miss, but it costs an API call per evaluation, adds latency, and can favour answers that sound stylistically like its own outputs.

DAY 39 · BUILDING WITH APIS (OPENAI, GEMINI, CLAUDE)

THE CORE IDEA

Calling an LLM API looks simple: send text, get text back. Four problems show up fast in production: you don't know a call's cost until after you count tokens, every API has rate limits that throw errors without retry logic, asking for JSON does not guarantee you get clean JSON back, and the same question asked repeatedly costs money every single time without caching.

API PRICING SNAPSHOT (PER 1K TOKENS, MID-2024)

Model	Provider	Input	Output
gpt-3.5-turbo	OpenAI	$0.0015	$0.002
gpt-4o	OpenAI	$0.005	$0.015
gpt-4	OpenAI	$0.03	$0.06
claude-3-haiku	Anthropic	$0.00025	$0.00125
claude-3-sonnet	Anthropic	$0.003	$0.015
gemini-1.5-flash	Google	$0.00035	$0.00105

Always check the official pricing pages before budgeting. These change often.

EXPONENTIAL BACKOFF, ATTEMPT BY ATTEMPT

```
wait = min(base * 2^attempt + random_jitter, max_wait)
Attempt 0: ~1s   Attempt 1: ~2s   Attempt 2: ~4s
Attempt 3: ~8s   Attempt 4: ~16s  Attempt 5: ~32s (capped)
```

Fixed-interval retry causes a thundering herd: every failed client retries at the same instant and trips the rate limit again. Exponential backoff with jitter spaces retries out so the API actually recovers.

SEMANTIC CACHING IMPACT

A 65% cache hit rate on GPT-3.5-turbo cuts the cost curve to roughly a third of the no-cache line as monthly call volume grows. Shopify reported a 40% API cost reduction using this exact pattern.

REAL WORLD PROBLEM · THE $8,000 API BILL

A startup's API cost went from $800/month to $8,000/month between January and March with no real growth in users. Three causes: no caching (the support bot re-answered the same 200 questions daily), untrimmed conversation history (by message 15, each call carried 8,000 tokens of mostly irrelevant context), and GPT-4 used for every call, including simple tasks GPT-3.5 handled equally well at 20x lower cost.

The fix: semantic cache (65% hit rate, saved $585/month), conversation trimming to the last 8 turns (cut average tokens per call by 60%), and model routing (simple tasks to GPT-3.5, complex reasoning to GPT-4, cutting the remaining cost by 40%). Combined: the bill dropped from $8,000 back to $1,100. Same product, same users, same quality.

GOLDEN RULE

LiteLLM gives you one completion() function across OpenAI, Anthropic, Gemini, and 100+ other providers, so switching providers means changing a model string, not rewriting your integration. Combine it with built-in fallbacks so a failed primary provider automatically routes to a backup, plus cost tracking and rate-limit handling out of the box.

BONUS · MNC INTERVIEW Q&A

14 questions pulled from documented interview patterns across Days 33-39. Read the question, answer it out loud before reading the direction.

DAY 33 · Q1 · Meta – ML Engineer Round

Why does GPT-4 use BPE tokenization instead of word-level tokenization?

Three reasons. Word tokenizers fail on anything outside the training vocabulary, discarding the entire word as [UNK]. Word vocabularies also grow fast: English alone has 170,000+ words, and GPT-4 covers 100+ languages plus code. BPE covers all of it with roughly 100K tokens, and falls back to character-level pieces for genuinely unknown strings, so it can tokenize an unusual function name even if it never appeared in training.

DAY 33 · Q2 · General MLE Round

Two sentences with no words in common score 0.95 on cosine similarity. How?

Embeddings capture distributional meaning, not word identity. A model trained on billions of sentences learns that words like 'dog' and 'canine' appear in similar contexts and play similar grammatical roles, pulling their vectors close together. Two sentences built from different but semantically equivalent words end up with near-identical embeddings despite sharing zero tokens.

DAY 34 · Q1 · General MLE Round

A product manager asks whether to fine-tune or prompt engineer. What do you ask first?

Four questions. How many labelled examples exist? Fewer than 100 makes fine-tuning risky regardless of anything else. Does the output need a strict format? If yes, fine-tuning is more reliable. How often does the task definition change? Frequent change makes fine-tuning expensive to maintain. What's the monthly call volume? Below roughly 20K calls/month, cost savings rarely justify the training cost. Only after these four does a recommendation make sense.

DAY 34 · Q2 · General MLE Round

You fine-tune on 500 examples, hit 95% on your test set, but only 71% in production. What happened?

Two likely causes. Overfitting: 500 examples is marginal, and the model may have memorised training patterns instead of generalising, especially if train and test came from the same narrow source. Distribution shift: production inputs may look structurally different from training examples. Sample 50 production failures and compare them to training data. If they diverge, expand the training set to cover production-like inputs, or fall back to a stronger prompted base model.

DAY 35 · Q1 · Google – ML Engineer Round

Why does RAG outperform fine-tuning when the underlying knowledge changes frequently?

Fine-tuning bakes knowledge into the weights, so updating it means retraining, which is slow, costly, and risks catastrophic forgetting. RAG stores knowledge externally in a vector database, so updating it means re-embedding the changed documents, a job that takes minutes. For a company updating documentation weekly, RAG is the only workable option. Fine-tuning is for changing model behaviour, not for keeping facts current.

DAY 35 · Q2 · Microsoft – ML Engineer Round

An answer needs combining information from three chunks that were never retrieved. What's wrong, and how do you fix it?

Single-round retrieval finds chunks similar to the query embedding. If the answer requires synthesising three separate pieces of information that don't individually look similar to the query, standard retrieval misses all three. Fixes: raise top-k to accept more noise, run multi-hop retrieval (use the first result to generate a follow-up query and retrieve again), or move to a graph-based retrieval approach like Microsoft's GraphRAG, which follows relationships between chunks instead of ranking them independently.

DAY 36 · Q1 · General MLE Round

What is the ReAct framework, and why does writing reasoning out before acting improve accuracy?

ReAct interleaves reasoning steps (Thought) with action steps (Action/Observation) instead of jumping straight to a tool call. Writing the reasoning first forces the model to commit to a plan before acting, and each reasoning step becomes part of the context for the next generation. This helps most on multi-step tasks, where each step builds on a verified prior conclusion instead of compressing several steps of reasoning into one high-error prediction.

DAY 36 · Q2 · Swiggy – ML Engineer Round

An agent picks the wrong tool three times in a row. What's the most likely root cause?

Almost always the tool descriptions. The model reads them to decide which tool to call, so if two tools have similar descriptions, or neither states what it does not handle, the model will guess wrong. Fix: rewrite descriptions with negative constraints, such as 'use for math only, not for factual lookups,' and make tool names specific rather than vague labels like 'search.'

DAY 37 · Q1 · General MLE Round

Why can't you use a regular SQL database for similarity search?

SQL databases index for exact match and range queries. Finding the 10 most similar rows by cosine similarity requires computing the distance from every row to the query and sorting, a full table scan every time, since similarity isn't a monotone function that tree indexes can prune. pgvector partially closes this gap by adding HNSW indexing inside Postgres, but dedicated vector databases are still significantly faster at real production scale.

DAY 37 · Q2 · Amazon – ML Engineer Round

You need 50M vectors searched at 99% recall and under 10ms latency. Which FAISS index, and how do you configure it?

IndexFlatIP is out on memory alone at this scale (roughly 72GB RAM for 50M 384-dim vectors). IndexIVFPQ compresses each vector to 48-96 bytes with product quantization, keeping memory under 5GB at 95-98% recall. For 99% recall specifically: IndexIVFFlat with nlist=4096, nprobe=256 on GPU, or HNSW with efSearch=256 on CPU. Profile both against your actual data distribution, since the optimal configuration varies by dataset.

DAY 38 · Q1 · General MLE Round

Why do traditional overlap metrics like ROUGE penalize valid LLM outputs?

ROUGE scores word and n-gram overlap against a single reference answer. An LLM response that is factually correct but phrased differently, using synonyms or a different sentence structure, scores lower than a close paraphrase even though both are equally valid. This is exactly why the field moved toward semantic similarity and LLM-based judging for generative text.

DAY 38 · Q2 · General MLE Round

What's the difference between faithfulness and answer relevancy, and why check both?

Faithfulness checks the answer against the retrieved context: is every claim grounded in it? Relevancy checks the answer against the question: does it actually address what was asked? An answer can be perfectly faithful and still useless if it answers the wrong question, or relevant but fabricated. Production systems check both because each catches a failure mode the other misses.

DAY 39 · Q1 · General MLE Round

Your monthly API cost doubled with no growth in users. What do you check first?

Three things, in order. Average input tokens per request over time; if it's growing, prompts or conversation history are bloating. Cache hit rate; if it dropped, you're re-answering questions you already answered. Model distribution per request; if more traffic silently shifted to a premium model, cost multiplies immediately. Logging these three metrics as dashboards catches most cost anomalies before they become an $8,000 surprise.

DAY 39 · Q2 · General MLE Round

Why does exponential backoff with jitter beat fixed-interval retry?

Fixed-interval retry causes a thundering herd: if 100 clients hit a rate limit simultaneously and all retry after exactly 1 second, they collide again on the retry. Exponential backoff makes each subsequent wait longer, giving the server room to recover, and jitter (a small random delay) spreads retries out even when many clients fail at once. Every major API provider recommends this combination as the standard retry pattern.

WEEK 5 · ALL QUESTIONS AT A GLANCE

Day · Company	Question	Key Points
33 · Meta	Why BPE over word-level tokenization?	Handles OOV, keeps vocab small, falls back to characters for unknown strings.
33 · General	0.95 similarity, zero shared words?	Embeddings encode distributional meaning, not word identity.
34 · General	Fine-tune or prompt engineer?	Check data volume, format strictness, task volatility, call volume.
34 · General	95% test, 71% production?	Overfitting on small data, or distribution shift. Sample failures and compare.
35 · Google	RAG vs fine-tuning for changing knowledge?	RAG updates in minutes via re-embedding. Fine-tuning requires retraining.
35 · Microsoft	Answer needs 3 unretrieved chunks?	Raise top-k, use multi-hop retrieval, or move to GraphRAG.
36 · General	Why does ReAct improve accuracy?	Reasoning before acting commits to a plan and conditions the next step.

Day · Company	Question	Key Points
36 · Swiggy	Wrong tool picked 3x in a row?	Vague tool descriptions. Add negative constraints and specific names.
37 · General	Why not SQL for similarity search?	Full table scan every query. No index structure prunes similarity.
37 · Amazon	50M vectors, 99% recall, <10ms?	IndexIVFPQ or HNSW with tuned efSearch/nprobe. Profile on real data.
38 · General	Why does ROUGE penalize valid outputs?	Overlap-based, punishes correct paraphrasing that differs from reference.
38 · General	Faithfulness vs relevancy?	Faithfulness checks against context. Relevancy checks against the question.
39 · General	Cost doubled, no user growth?	Check token count trend, cache hit rate, and model routing distribution.
39 · General	Why backoff + jitter over fixed retry?	Prevents thundering herd; spreads retries so the API can recover.

SELF-TEST BEFORE WEEK 6

Without notes: explain why you'd store source text alongside an embedding, not just the vector. State the difference between faithfulness and answer relevancy in a RAG system. Describe two guards that stop an LLM agent from looping indefinitely. Explain why IndexFlatIP works at 100K vectors but fails at 10 million. If any of these feel shaky, go back to that day before moving on.

Week 6 starts with Production: Deploying an ML Model with FastAPI, then MLOps Basics and the full end-to-end capstone project.

Full notebooks: github.com/VaishnaviJagtap18/42-days-aiml-challenge · **#42DaysOfML**